CIS*3110 Operating Systems

Week 10

Lecture #15 Implementing locks

Prof. Yulia Kotseruba

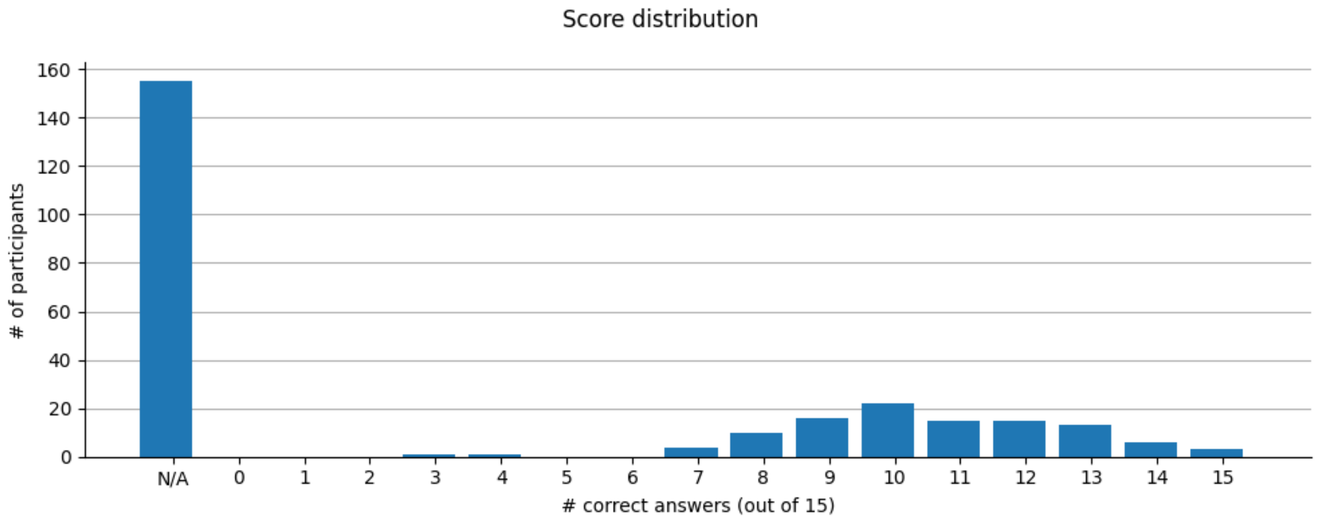
March 10, 2026

Quiz 13

```
static int n = 0;
void inc(void) {
        printf("%d\n", ++n);
}

int main(void) {
    inc(); fork(); inc();
}
```

- **71%** average (up from 61% in quiz 12)
- 3 hardest questions:
  - (36%) What will be printed when this code executes? a) 1 1 1 b) 1 2 2 c) 1 2 3 d) 1 1 2
  - (42%) What process transition is not possible? a) ready -> waiting b) running -> ready c) running -> terminated d) waiting -> ready
  - (46%) What will be printed when this code executes? a) ABC b) ACB c) AC d) AB



n=1 after inc() -> prints 1

after fork() both processes have n=1

after inc() both processes have n=2

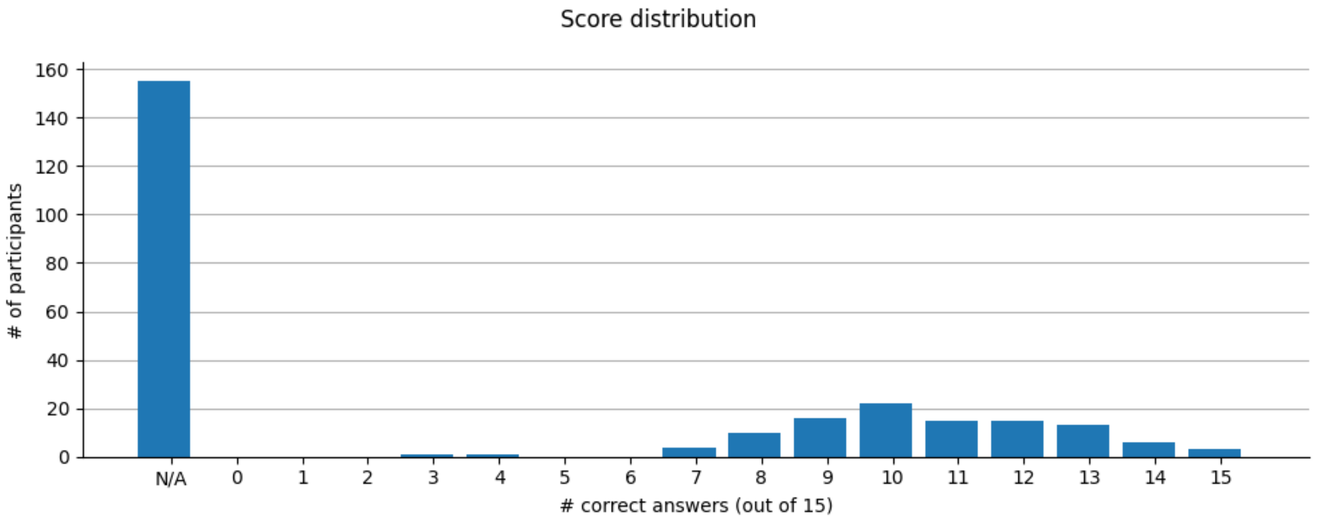
both will print 2 (order doesn't matter)

Quiz 13

- **71%** average (up from 61% in quiz 12)
- 3 hardest questions:
  - (36%) What will be printed when this code executes? a) 1 1 1 b) 1 2 2 c) 1 2 3 d) 1 1 2
  - (42%) What process transition is not possible? a) ready -> waiting b) running -> ready c) running -> terminated d) waiting -> ready
  - (46%) What will be printed when this code executes? a) ABC b) ACB c) AC d) AB

```
static int n = 0;
void inc(void) {
        printf("%d\n", ++n);
}

int main(void) {
    inc(); fork(); inc();
}
```




```
int main(void) {
    printf("A");
    if (fork() == 0) {
        sleep(1);
        execlp("echo", "echo", "B", NULL);
    }
    printf("C");
}
```

child will print B after sleeping

parent will accumulate AC in its buffer
it will be flushed after main exits

Recap

- **Scheduling threads**
  - depends on the multithreading model
    - can be scheduled as a group or individually
  - difficult to predict when each thread will run -> **do not make assumptions**
- **Race condition** - when order of thread execution changes the results
- **Critical section** - part of the code that can be affected by race condition
  - usually because the operations are not atomic
- **Mutex lock** - protects the critical section from concurrent execution
- **Condition variable** - allows signalling to other threads that are waiting for a condition to be satisfied

Lock components

- state of the lock
  - available or acquired (usually a variable)
- which thread is holding it
- `lock()` and `unlock()` functions

Building a lock

- `lock()`
  - if the lock is free
    - the thread will acquire it and enter the critical section
    - this thread is now the **owner** of the lock
      - if more than one thread calls lock, only **one** gets it
      - scheduler decides which one
  - otherwise
    - the thread will be blocked until the owner releases it
- `unlock()`
  - if called by the owner, releases the lock
  - if called by another thread (often undefined behavior)

Lock features

- **mutual exclusion** - guarantee that only one thread is in the critical section
- **fairness** - every waiting thread should have equal chance of getting a lock
  - in other words, avoid starvation
- **performance** - overhead from using the lock

Naive locks

- **Option #1 Disable interrupts**
  - process inside the critical section will not be interrupted
    - all operations will act as atomic
  - pros:
    - simple
  - cons:
    - will not work in multi-processor systems
    - turning off interrupts is dangerous
      - OS cannot regain control if needed (e.g. when infinite loop happens)
      - other interrupts may be lost (e.g. from keyboard/hard drive)

```
void lock() {
  disable_interrupts();
}
void unlock() {
  enable_interrupts();
}
```

Naive locks

- **Option #2 Global variable**
  - flag = 0 - lock is free
  - flag = 1 - lock is held
  - lock()
    - continuously test if flag is available
    - set flag = 1 to indicate ownership
  - unlock()
    - set flag = 0

```
int flag = 0;

void lock() {
  while(flag == 1) flag = 1;
}
void unlock() {
  flag = 0;
}
```

Naive locks

- **Option #2 Global variable**
  - flag = 0 - lock is free
  - flag = 1 - lock is held
  - lock()
    - continuously test if flag is available
    - set flag = 1 to indicate ownership
  - unlock()
    - set flag = 0

```
int flag = 0;

void lock() {
  while(flag == 1) flag = 1;
}
void unlock() {
  flag = 0;
}
```

called spin lock because the waiting thread "spins" while lock is being used

```c
//includes omitted

int flag = 0;
int counter = 0;

void lock()   { while (flag); flag = 1; }
void unlock() { flag = 0; }

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        lock();
        counter++;
        unlock();
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

lock waits for the flag to be 0, then sets it to 1

releases the lock

lock the critical section

create two threads that will increment the counter

simple_lock.c

```c
//includes omitted

int flag = 0;
int counter = 0;

void lock()   { while (flag); flag = 1; }
void unlock() { flag = 0; }

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        lock();
        counter++;
        unlock();
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

simple_lock.c

let's test the lock

```
>>> ./simple_lock
counter = 1325
```

```
>>> ./simple_lock
counter = 1527
```

```
>>> ./simple_lock
counter = 2000
```

the lock doesn't seem to work

why?

checking and setting the flag is not an atomic operation

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A starts first

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A starts first

checks the flag

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A is interrupted

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A is interrupted

B starts

thread A

flag = 0

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A is interrupted

B starts

checks the flag

thread A

flag = 1

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A is interrupted

B starts

checks the flag

sets it to 1

thread A

flag = 1

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A is interrupted

B is interrupted

thread A

flag = 1

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A starts again

B is interrupted

thread A

flag = 1

thread B

```
void lock() {
  while(flag == 1)
  flag = 1;
}
```

A sets flag to 1

B is interrupted

now both threads can go into the critical section and increment the counter

counter++ is not atomic -> race condition (see Lecture #14)

Peterson’s algorithm

- An algorithm for mutual exclusion developed by Gary Peterson in 1981
  - for 2 or more threads
  - using only shared memory for communication
    - flag variable
      - flag[n] = true — process n wants to enter the critical section
    - turn — whose turn it is to go to the critical section

Peterson’s algorithm

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

- An algorithm for mutual exclusion developed by Gary Peterson in 1981
  - for 2 or more threads
  - using only shared memory for communication
    - flag variable
      - flag[n] = true — process n wants to enter the critical section
    - turn — whose turn it is to go to the critical section

```
void unlock(int me){
  flag[me] = 0
};
```

Peterson’s algorithm for 2 thread mutex

Peterson’s algorithm

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

- Assume 2 threads
  - flag[0]==1 — P0 wants to enter CS
  - flag[1]==1 — P1 wants to enter CS
  - turn == 0 - P0 is allowed to proceed if flag[0] = 1
  - turn == 1 - P1 is allowed to proceed if flag[1] = 1

```
void unlock(int me){
  flag[me] = 0
};
```

Peterson’s algorithm for 2 thread mutex

Peterson’s algorithm

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

- Possibilities
  - flag[0] = flag[1] = false — no thread wants to enter CS
  - one of flag entries is false — only one thread wants to enter CS, turn value is irrelevant
  - flag[0] = flag[1] = true - both threads want to enter CS
    - turn will determine which one has priority
- Both threads cannot enter CS at the same time
  - the while condition is (flag[other] && turn == other)
  - the following needs to be satisfied for both threads to break out of it
    - flag[0] = flag[1] = true
    - turn = 0 and turn = 1
  - these cannot happen at the same time, therefore mutual exclusion is guaranteed

```
void unlock(int me){
  flag[me] = 0
};
```

Peterson’s algorithm for 2 thread mutex

Peterson's algorithm for 2 thread mutex

```
//includes omitted

volatile int flag = {0, 0};
volatile int turn = 0;
int counter = 0;

void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}

void unlock(int me){flag[me] = 0};

void *worker(void *) {
    int me = (int) arg;
    for (int i = 0; i < 1000; i++) {
        lock();
        counter++;
        unlock();
    }
    return NULL;
}
```

flag indicates that thread i wants the lock

turn indicates which thread will give the lock if both are waiting

thread sets its flag to 1 and change turn to the other thread

wait for its turn

unlocks by resetting its flag

the main is the same

the only difference is passing thread_id as an argument to pthread_create

```
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, 0);
    pthread_create(&t2, NULL, worker, 1);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

the worker is the same

flag = {0, 0}

turn = 0

thread A

me = 0

thread B

me = 1

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

flag = {0, 0}

turn = 0

thread A

me = 0

other = 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

flag = {1, 1}

turn = 0

thread A

me = 0

other = 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

flag = {1, 1}

turn = 0

thread A

me = 0

other = 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

flag = {1, 1}

turn = 1

thread A

me = 0

other = 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

even if both reach this line at the same time

whoever writes last will win

say B writes last

flag = {1, 1}

turn = 1

thread A

me = 0

other = 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

true

true

true

false

A will stay in the while loop

B will break out of the while loop and get the lock

```c
//includes omitted

volatile int flag = {0, 0};
volatile int turn = 0;
int counter = 0;

void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}

void unlock(int me){flag[me] = 0};

void *worker(void *) {
    int me = (int) arg;
    for (int i = 0; i < 1000; i++) {
        lock();
        counter++;
        unlock();
    }
    return NULL;
}
```

peterson_lock.c

Peterson's algorithm for 2 thread mutex

```
>>> ./peterson_lock
counter = 2000
```

```
>>> ./peterson_lock
counter = 1998
```

not again! why is it not working?

many reasons, in short, modern hardware, OSs and compilers are too complex

volatile prevents caching but does not enforce writes

writes may happen out of order

cores may not be synced in time

```c
int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, 0);
    pthread_create(&t2, NULL, worker, 1);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

CPU core 0

thread A

me = 0

other = 1

flag = {0, 0}

turn = 0

CPU core 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

CPU core 0

thread A

me = 0

other = 1

buffer flag[0] = 1

flag = {0, 0}

turn = 0

CPU core 1

thread B

me = 1

other = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

A changes flag[0] but it stays in its buffer

CPU core 0

thread A

me = 0

other = 1

buffer flag[0] = 1

flag = {0, 0}

turn = 0

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

CPU core 1

thread B

me = 1

other = 0

flag[1] = 1 buffer

A changes flag[0] but it stays in its buffer

B changes flag[1] but it stays in its buffer

CPU core 0

thread A

me = 0

other = 1

buffer flag[0] = 1

flag = {0, 0}

turn = 1

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

CPU core 1

thread B

me = 1

other = 0

flag[1] = 1 buffer

A updates turn and it gets written

CPU core 0

thread A

me = 0

other = 1

buffer flag[0] = 1

flag = {0, 0}

turn = 1

CPU core 1

thread B

me = 1

other = 0

flag[1] = 1 buffer

```
void lock(int me){
    int other = 1 - me;
    flag[me] = 1;
    turn = other;
    while (flag[other] && turn == other);
}
```

false	true
false	true

A and B have not written their flag values yet so they cannot see each other's changes both will break out of the loop and enter the critical section

Peterson’s algorithm

- How good is it?
  - **Correctness** - not great (breaks on modern systems)
  - **Fairness** - not great
    - no process will wait forever or become deadlocked
    - thread order/timing depends on the CPU scheduler
  - **Performance** - not great
    - busy-waiting - all waiting threads use CPU

Summary so far

- Naive locks don't work
  - Option #1 **turning off interrupts** is too dangerous
  - Option #2 **spin lock** with one global variable has a race condition
  - Option #3 **Peterson's software mutex** does not work on modern systems
- **We need hardware support!**
  - Option #1 test-and-set and spin
  - Option #2 test-and-set and yield
  - Option #3 futex
  - there are more options, read in Chapter 27

Test-and-set

- Test-and-set hardware instruction
  - takes value addr and a new value
  - tests the old value at the address
  - sets the address to a new value
    - both operations are done **simultaneously**
  - returns the old value

```
func test_and_set(addr, new)
  old = *addr
  *addr = new
  return old
```

```c
//includes omitted

int flag = 0;
int counter = 0;

void lock()   { while (test_and_set(&flag, 1) == 1); }
void unlock() { flag = 0; }

void *worker(void *) {
    for (int i = 0; i < 1000; i++) {
        lock();
        counter++;
        unlock();
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, worker, NULL);
    pthread_create(&t2, NULL, worker, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("counter = %d\n", counter);
    return 0;
}
```

ts_lock.c

this is our naive spin lock from before

if we were implementing the lock in the kernel, we would call on something like test_and_set in the lock() function

this is a proper functioning lock

are we done?

```
>>> ./ts_lock
counter = 2000
```

```
>>> ./ts_lock
counter = 2000
```

```
>>> ./ts_lock
counter = 2000
```

Spin lock

- What we implemented is called a **spin lock**
- How good is it?
  - **Correctness** - OK!
  - **Fairness** - not starvation-free
    - CPU scheduler controls which thread acquires the lock next
    - cannot guarantee that the waiting thread will not wait forever
  - **Performance** - not great
    - busy-waiting - all waiting threads use CPU

Lock with yield

```
void lock() {
  while (test_and_set(&flag, 1) == 1)
    yield();
}
```

- **Yield instead of spinning**
  - waiting thread will **deschedule itself** and let others run
- **Correctness** - OK
- **Fairness**
  - CPU scheduler controls which thread acquires the lock next
  - not starvation-free - a thread can enter an endless yield loop if other threads get the lock first
- **Performance** - better than spinning but still not great

Lock with yield

```
void lock() {
  while (test_and_set(&flag, 1) == 1)
    yield();
}
```

- **Yield instead of spinning**
  - waiting thread will **deschedule itself** and let others run
- **Correctness** - OK
- **Fairness**
  - CPU scheduler controls which thread acquires the lock next
  - not starvation-free - a thread can enter an endless yield loop if other threads get the lock first
- **Performance** - better than spinning but still not great
  - assume 100 threads compete for the lock and RR CPU scheduler
    - 1 thread will get the lock
    - 99 threads will execute run-and-yield
    - context switching costs will add up

problem #1 need to keep track of threads holding the lock

problem #2 need to minimize kernel involvement

Futex

- **Futex** - **f**ast **u**serlevel mu**tex** (in most Linux kernels since v2.5.7)
  - kernel-level wait queue attached to user-level atomic integer
    - 32-bit lock variable in the userspace
  - threads interact through the integer in userspace
  - kernel calls needed for operations on the queue
    - only in the case when multiple threads compete for the lock

Franke, Russel, & Kirkwood, *Fuss, Futexes, and Furwocs: Fast Userlevel Locking in Linux*, Ottawa Linux Symposium, 2002

```
void futex_lock(int *futex){
  int v;
  if (atomic_bit_test_set(futex, 31) == 0)
    return;

  atomic_increment(futex);

  while(1) {
    if (atomic_bit_test_set(futex, 31) == 0) {
      atomic_decrement(futex);
      return;
    }
    v = *futex;
    if (v >= 0) continue;
    futex_wait(futex, v);
  }
}

void futex_unlock(int *futex) {
  if (atomic_add_zero(futex, 0x8000000))
    return;

  futex_wake(futex);
}
```

futex is a 32-bit integer in the user space

bit 31 indicates whether the lock is used

set bit 31 of futex, if it was 0, the caller now owns the lock

otherwise increment the lower 31 bits of futex -> counter for waiting threads

try to acquire the lock

if the lock is not held

decrement the lower bits to show that fewer threads are waiting

otherwise check the value of the futex again

this means that the lock is not held, but threads may be waiting

otherwise, the lock is held and there may be threads waiting

put the thread to sleep

this is 0x100….000 (31 zeros)

atomic_add_zero adds and checks if the value is 0

if futex + 0x80000000 is 0, no thread is waiting

otherwise, there are threads waiting, wake up the next one

futex_lock.c

Futex

- Futex - fast userlevel mutex
  - kernel-level wait queue attached to user-level atomic integer
    - 32-bit lock variable in the userspace
  - threads interact via the integer in userspace
  - kernel calls needed for operations on the queue
    - only in the case when multiple threads compete for the lock
- Correctness - OK
- Fairness - OK
  - rely on a queue rather than the whims of the CPU scheduler
- Performance - OK
  - few operations, most of which are done in userspace
  - minimizes context switching

Summary so far

- Option #1 test-and-set and spin
  - test-and-set checks the value, sets it to a new one and returns the old one
  - not fair, not starvation-free, spinning wastes CPU
- Option #2 test-and-set and yield
  - not fair, not starvation-free, less overhead
- Option #3 futex
  - **fair** (uses a queue), **starvation-free**, **efficient** (one variable, few operations, minimizes kernel calls)
  - futex is what modern Linux kernels are using for implementing mutexes, conditional variables, and semaphores

Condition variables

- Locks alone are not enough
- Often also need to check if a condition has been satisfied

```c
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (!done) ;
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

thread prints and updates a variable

main prints

main creates a thread

main spins waiting for the variable to be updated

main prints and exits

```
>>> ./threads_talking
parent: begin
child
parent: end
```

this works but it's inefficient -> main thread spins wasting CPU resources

Condition variables

- Introduced by Dijkstra in 1968
- Implemented as a queue
  - holds threads waiting on the condition
  - releases them when the condition is satisfied

Condition variables

- In pthreads library, each condition variable is associated with the lock
  - `pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)`
    - puts the calling thread to sleep
    - unlocks the mutex
    - waits for the other thread to signal it
    - when signalled, reacquires the lock (may need to wait)
- When something changes about the condition
  - `pthread_cond_signal(pthread_cond_t *cond)`
    - unblocks at least one thread blocked by condition
    - if more than one thread is blocked, the order is decided by the scheduler

this is the same code
with condition var

```
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (!done) ;
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

thr_exit sets sets
done=1 and signals

thr_join waits for the
signal

```c
volatile int done = 0;

void *child(void *arg) {
    printf("child\n");
    done = 1;
    return NULL;
}

int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    while (!done) ;
    printf("parent: end\n");
    return 0;
}
```

threads_talking.c

```c
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

2 possibilities:

child thread executes child() first

main thread executes thr_join()

main

done = 0

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```


case #1 child executes first

main

done = 0

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child thread executes first

case #1 child executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child thread will obtain a lock

set done to 1

signal and return

there are no threads waiting for the signal but it's ok

case #1 child executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```


case #1 child executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

main gets the lock

```
void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}
```

```
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```


case #1 child executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

main gets the lock

checks done and releases the lock

case #1 child executes first

main

done = 0

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```


case #2 main executes first

main

done = 0

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

main will go into the while loop

case #2 main executes first

main

done = 0

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

main will go into the while loop

wait will put it to sleep and release the lock

case #2 main executes first

main

done = 0

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
```

child

```
void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}
```

child will set done to 1

```
void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```


case #2 main executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child will set done to 1

signal and release the lock

case #2 main executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

main will wake up, re-acquire the lock

case #2 main executes first

main

done = 1

child

```
volatile int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

check done and release the lock

case #2 main executes first

main

~~done = 0~~

child

```
~~volatile int done = 0;~~
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  done = 1;
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  while (done == 0)
    pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

can we do this without the done variable?

main

child

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child goes first and signals but there are no threads waiting

main

child

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child releases the lock and exits

main

child

parent will get the lock

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

main

child

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

wait will put it to sleep and release the lock

main

child

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  pthread_mutex_lock(&m);
  pthread_cond_signal(&c);
  pthread_mutex_unlock(&m);
}

void thr_join() {
  pthread_mutex_lock(&m);
  pthread_cond_wait(&c, &m);
  pthread_mutes_unlock(&m);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

wait will put it to sleep and release the lock

it will sleep forever because child will never wake it…

main

done = 0

child

```
volatile int done = 0;
~~pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;~~
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  ~~pthread_mutex_lock(&m);~~
  done = 1;
  pthread_cond_signal(&c);
  ~~pthread_mutex_unlock(&m);~~
}

void thr_join() {
  ~~pthread_mutex_lock(&m);~~
  if (done == 0)
    pthread_cond_wait(&c, ~~&m~~);
  ~~pthread_mutes_unlock(&m);~~
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

change while to if

pretend that conditional variable does not require a lock

can we do this without locks?

main

done = 0

child

```
volatile int done = 0;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}

void thr_join() {
  if (done == 0)
    pthread_cond_wait(&c);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

parent checks done

main

done = 1

child

```
volatile int done = 0;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}

void thr_join() {
  if (done == 0)
    pthread_cond_wait(&c);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child updates done

parent checks done
but is interrupted right after

main

done = 1

child

```
volatile int done = 0;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}

void thr_join() {
  if (done == 0)
    pthread_cond_wait(&c);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

child updates done

and signals, but no one is waiting

parent checks done

but is interrupted right after

main

done = 1

child

```
volatile int done = 0;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;

void thr_exit() {
  done = 1;
  pthread_cond_signal(&c);
}

void thr_join() {
  if (done == 0)
    pthread_cond_wait(&c);
}

void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(void) {
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

parent wakes up
and waits forever…

Condition variables

- **Needs both locks and global variables to hold state**
- Rule of thumb
  - hold the lock when signalling
    - avoids race conditions
  - hold the lock when waiting
    - enforced by pthread_cond_wait
      - assumes the lock is held
      - releases the lock and puts the caller to sleep
      - re-acquires the lock before returning

Summary

- Software-only locks do not work on modern hardware
- Atomic hardware instructions are the standard
  - Option #1 test-and-set and spin - not fair, not starvation-free, spinning wastes CPU
  - Option #2 test-and-set and yield - not fair, not starvation-free, less overhead
  - Option #3 futex - fair (uses a queue), starvation-free, efficient (one variable, few operations, minimizes kernel calls)
- Condition variables - useful for communication
  - require both locks and global state to work correctly

Reading

- Chapter 28 Locks
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-locks.pdf
- Chapter 30 Condition variables
  - https://pages.cs.wisc.edu/~remzi/OSTEP/threads-cv.pdf
- Code
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-locks
  - https://github.com/remzi-arpacidusseau/ostep-code/tree/master/threads-cv

Quiz 14

- This quiz **is graded**
- Your Kahoot! nickname **must be your student id**
  - e.g. 9999999
- Use any device, e.g. phone, laptop, tablet
- If you get disconnected and cannot use the same id, append underscore (_) before the id, e.g. _9999999